The Guarantees Hold, the Costs Diverge

A Two-Sided Conformance and Cost-Ledger Study of Vector Databases for Governed Agent Memory

Camilo Ayerbe Posada — GNS Foundation

June 12, 2026 — draft v1.0 (Zenodo deposit)

Abstract

The GRAFOMEM benchmark established that an advertised memory capability is not a guarantee: reference backends that claim hard-delete or tenant scoping, accept every call, and report success can leak completely — while explicitly leaving open whether production vector databases exhibit the same failure. This paper answers that question. We drive two widely deployed engines (pgvector, Qdrant) through the benchmark's two-sided deletion and tenancy workloads via an oracle-grounded harness, in two postures: *engine-correct* (the scoping predicate applied as documented) and *engine-as-deployed* (the predicate omitted, as a single integration fault would). Across three orders of corpus magnitude (10^4–10^6 vectors) and supersession depths to 16, driven as documented, the engines are honest: deletion is enforced on the read path at the first probe, scoped retrieval never leaks, and claims-but-leaks never reproduces. The failure mode is real — one layer up: in the deployed posture both engines silently return cross-tenant results at 0.85 leakage with full-looking output and no recall footprint. What separates the engines is the *price* of correctness, which we meter with a cost ledger pairing every verdict with its cost on a single workstation fingerprint at dim=64: post-filter architectures starve under selective filters by a mechanism we confirm with three independent instruments; planning architectures hold exactness cheaply, but the planner is load-bearing — forcing traversal collapses recall to zero, the study's only over-restriction verdict; and version-as-row supersession taxes recall twice. Every measured gap closes at the adapter layer, including one 30–40× "engine cost" that a registered probe revealed to be our own instrument's missing index — an episode we report in full as evidence for the methodology. We conclude that guarantee verification belongs in deployment pipelines, not only engine test suites.

Contents

Artifact: DOI 10.5281/zenodo.20666339 (v1.0.0) · concept DOI 10.5281/zenodo.20666338 · github.com/cayerbe/grafomem-db-bench. Every numeric claim resolves to a hashed ledger row (AUDIT_RESOLVED.md, 28/28); inline [F-...] tags reference the evidence log. Companion benchmark: GRAFOMEM (academia.edu/167598244).

1 Introduction

Agent memory systems — the stores that let a language-model agent remember, revise, and forget across sessions — are overwhelmingly built on vector databases, and the guarantees those systems advertise to users (your data is isolated; your deletion is honored) are therefore, in the end, claims about what a vector database returns. The companion benchmark to this paper [GRAFOMEM] showed how fragile such claims can be in principle: it exhibited reference backends that declare hard-delete or multi-tenancy, accept every relevant call, return success — and leak at 1.000, because the read path never consults the tombstone or the scope. It also drew its own scope limit in ink: the failure *mode* was proven on purpose-built foils, and whether any commercial engine exhibits it was explicitly unobserved. To our knowledge no one had run a deterministic, oracle-grounded, two-sided privacy conformance suite against production vector databases. This paper does, and reports what it found together with what it cost to find it.

Three findings structure the paper, and they arrive in an order the companion paper's reader will not expect. **First, the inversion: driven as documented, the engines are honest.** Across two engines spanning two filtering architectures, corpora from 4×10^3 to 10^6 vectors, and supersession depths to 16, deleted records were unrankable at the first post-delete read and scoped retrieval never returned an out-of-scope record. The claims-but-leaks failure mode did not reproduce on a single commercial configuration — including on the architecture where it was likeliest (mark-deleted points inside live HNSW segments with deferred reclamation). **Second, the relocation: the failure mode is real, one layer up.** In the *engine-as-deployed* posture — the tenant predicate omitted on read, the single most plausible integration fault — both engines silently return the global top-k: cross-tenant leakage 0.85–0.86, full-looking results, no error, and no recall footprint on the leaking side. Neither engine over-claims; both enforce exactly the predicate they are given. Tenancy on both is *by convention* — opt-in per query, defeatable by a one-line omission in a calling layer the engine never sees. The observable failure the companion paper located inside dishonest stores arises in production from honest stores driven by forgetful integrations, and a conformance practice that tests only the engine-correct path is structurally unable to see it. **Third, the divergence: what differs between engines, sharply, is the price of correctness.** A post-filter architecture starves under selective filters by a predictable mechanism — we confirm the candidate- survival model with the engine's own plan text, a forced-index probe, and a statistics-pinned ledger — and its

mitigation decays with scale; a planning architecture delivers exactness cheaply, but its cardinality planner is load-bearing: forcing graph traversal produced the study's only over-restriction verdict, recall 0.00 with leakage 0.00, the cell that justifies scoring both directions. Version-as-row supersession, the representation agent memory actually uses, taxes recall twice. And every one of these gaps — including the temporal one, the only gap that ever pointed toward purpose-built storage — closed at the adapter layer, each engine by a different one-screen fix, against gate criteria registered before the probes ran.

A fourth result is methodological and we give it a full section: one of our own headline numbers was wrong by more than an order of magnitude, the defect was in our adapter rather than the engine, and a registered competing-hypothesis probe caught it before publication (§6). We report the episode in detail because it is the paper's deployment-surface thesis in miniature — the engine was honest; the layer driving it was the fault — and because a benchmark whose own harness sits outside its audit scope will eventually publish its harness as an engine result.

Concretely, this paper contributes: (i) the first run, to our knowledge, of a two-sided, oracle-grounded agent-memory conformance suite against commercial vector databases, with both pass and fail directions reported; (ii) the *deployment-posture split* (engine-correct vs engine-as-deployed) as a methodological device, and the finding that the realistic privacy failure lives in the posture, not the engine; (iii) a verdict-paired cost ledger in which no guarantee is reported without its measured price, and no latency without the recall it bought; (iv) a mechanism account of post-filter starvation confirmed by three independent instruments, and a per-segment planner model that survived a failed prediction, refinement, and out-of-sample confirmation; (v) adapter-layer closures for every measured gap, with registered gates, including the demotion of native bi-temporality from requirement to optimization; and (vi) a fully documented self-caught confound as evidence that registered predictions and provenance-preserving ledgers are load-bearing parts of benchmark methodology rather than ceremony.

The paper proceeds as follows. Section 2 summarizes the inherited benchmark machinery and the two foil results this study carries to production. Section 3 specifies the method, including the posture split, the corpora, and the scope rules separating portable from fingerprint-bound claims. Section 4 presents the guarantee results and the inversion; Section 5 the cost results and the divergence; Section 6 the confound. Section 7 draws the implications for conformance practice, protocol design, and engine selection; Sections 8–10 give limitations, related work, and artifacts.

---

2 Background: the inherited benchmark and the two foils

GRAFOMEM [companion] is a deterministic, capability-typed benchmark for agent memory: facts carry bi-temporal validity and tenant ownership; an oracle derives, for every query, exactly which facts a correct system must return; independent validators gate every trace; and the corpus is content-addressed and regenerated byte-identically across architectures. Backends declare capabilities (HARD_DELETE, MULTI_TENANT, BI_TEMPORAL, …) as typed adapters, so what a system claims is explicit and separable from what it does, and the safety-critical boundaries are scored two-sided — leakage and over-restriction.

Two of its findings are this paper's premise. `soft_delete` claims HARD_DELETE, tombstones every deleted fact, reports success, and its own audit certifies the fact as gone — yet it leaks at 1.000, because retrieval never consults the tombstone (F10). `leaky_tenant` claims MULTI_TENANT, accepts a tenant scope on every call, tags every record's owner — and ranks across all tenants, leaking at 1.000 while holding in-tenant recall at 1.000, so the failure leaves no recall footprint (F12). Both satisfy the type contract and break the semantic one; only an external, oracle-grounded, two-sided check tells them apart. The companion paper drew its own scope limit explicitly: these are purpose-built foils, and whether commercial engines exhibit the failure was unobserved. That open question is this paper's charter, and the W5/W6 workloads — together with the verdict rule that 0.000 leakage alone is never a pass — are inherited unchanged.

---

3 Method

3.1 The inherited instrument

This study drives production engines through the two-sided deletion (W6) and tenant-isolation (W5) workloads of the GRAFOMEM benchmark [companion paper], preserving its three commitments. Correctness is *oracle-grounded*: every retrieved set is scored against an exact brute-force k-NN oracle computed over the same seed-pinned corpus the engine ingested — never against another engine's output, and never by a model judge. Scoring is *two-sided*: a guarantee fails by returning what it must not (leakage) or by withholding what it must (over-restriction), and a configuration that returns nothing earns OVER_RESTRICTS, not a pass; 0.000 leakage alone is never sufficient. And every verdict is *deterministic given the seeds*: recall, leakage, and the operation-count metrics reproduce exactly on any machine; only latency is environment-bound (§3.6).

The unit of measurement is a ledger row pairing a correctness verdict with the price of obtaining it: engine and version, the verbatim index configuration that produced the row (including the ANN search parameter — recall@k is meaningless without it), the verdict (PASS / PARTIAL / LEAKS / OVER_RESTRICTS), recall@k, leak rate, latency percentiles, build throughput, storage footprint, and for deletion the operation count until a deleted record is unrankable (`ops_until_gone`) — a deliberately hardware-independent W6 footprint: an honest read path goes at probe 1; a read path that never consults the tombstone never goes within budget.

3.2 The deployment posture split (7a / 7b)

The companion paper's claims-but-leaks finding concerns backends that advertise a capability and silently fail to enforce it. Testing whether production engines exhibit this requires a methodological decision that, to our knowledge, prior evaluations leave implicit: *which configuration of the engine is the unit of test?* An adapter that uses each engine's API exactly as documented measures the engine on its best behavior — and will systematically under-find the failure mode, because in deployed systems the scoping predicate is supplied per query by the calling layer, and the realistic failure is that layer omitting it.

We therefore run every probe in two postures and report them separately:

- **7a (engine-correct):** the API used as documented — tenant predicate applied, deletion semantics respected. This measures the engine.
- **7b (engine-as-deployed):** the single most plausible integration fault — the tenant predicate omitted on read, everything else identical. This measures the deployment surface the engine exposes.

A 7a-PASS together with a 7b-LEAK is not a contradiction; it is the finding that the guarantee exists but is opt-in per query and silently defeatable (§4.3). Conversely, only a 7a failure would indict the engine itself.

3.3 Corpora

All corpora are procedurally generated from pinned seeds (corpus 1729, queries 4104; query vectors drawn from a disjoint stream so no query is a corpus member), at dimension 64 with k=10 throughout — a deliberate conditioning choice discussed in §3.6 and §8.

Conformance corpus (W5/W6 probes): n = 4,000 vectors across 8 tenants whose cluster centers are interleaved in the embedding space, so that the true nearest neighbors of a tenant-A query include tenant-B vectors. This is the adversarial construction the isolation probe requires: a store can pass W5 only by excluding on *scope*, never by distance. The deletion probe removes records that are near neighbors of the probe query, so a removal visibly changes the correct answer.

Scale corpus (cost sweep): the same construction at $N \in \{2\times10^4, 10^5, 10^6\}$ with 64 tenants, fixing per-tenant selectivity at 1/64 — selective enough to stress filtered search, realistic for multi-tenant memory.

Supersession corpus (temporal stress): 20,000 logical facts, each a chain of $D \in \{1, 4, 16\}$ versions (20k–320k physical rows). Successive versions are near-duplicates (drift 0.15) — the realistic worst case, since a superseded fact's stale versions are exactly the vectors most likely to be confused with the current one. Version v of a chain is valid on the epoch $[v{\cdot}E, (v+1){\cdot}E)$ with $E = 100$; the last version is open-ended, encoded with a sentinel constant (valid_until = 2^{62}) so that "still valid" is an indexable predicate. Queries probe `as_of` at mid-history and inside the open interval ("now"), scored against an exact bi-temporal oracle; the leak metric counts temporally invalid versions among returned results.

3.4 Engines, adapters, configurations

Two engines were instrumented through a common adapter protocol (write / supersede / delete / retrieve / audit / flush, plus compact and storage_bytes), chosen as the locally-runnable representatives of two filtering architectures:

pgvector (PostgreSQL extension; HNSW, M=16, ef_construction=64, ef_search swept over {16, 40, 64, 128} in conformance runs and {16, 64, 256} at scale; versions recorded per row). Adapter hygiene that the study itself forced: `ANALYZE` is run after every load, because plan choice on a planner-mediated engine is a function of statistics state — without pinning, identical queries on identical data produced different verdicts across fresh instances [F-PG-7/8]. The adapter exposes the engine's own filtered-scan mitigation (`hnsw.iterative_scan = relaxed_order`) as a variant, and a forced-index mode for mechanism probes. Storage is measured exactly via `pg_total_relation_size`.

Qdrant (server, qdrant-client 1.18.0; HNSW, m=16, ef_construct=64, hnsw_ef swept identically). Two instrument lessons are part of the method record. First, Qdrant builds the vector index only above an indexing threshold; at n=4,000 the default left every segment on exact search, so early rows measured the plain-search path while appearing to measure HNSW [F-QD-1]. The adapter therefore sets `indexing_threshold=10`, waits for optimizer status green, and records `indexed_vectors_count` into every row's configuration string — which code path ran is observed per row, never inferred from recall shape. Second, a variant with `full_scan_threshold` forced low (10 KB) compels graph traversal, isolating the engine's planner from its index (§5.2). Qdrant does not expose exact on-disk size cheaply; its storage column is an estimate (points × dim × 4) and is tagged estimate-class, never compared against pgvector's exact figures.

3.5 Verdicts

The two-sided rule is mechanical: leak > 0 → LEAKS; leak = 0 with recall = 0 → OVER_RESTRICTS; leak = 0 with recall ≥ 0.999 → PASS; otherwise PARTIAL. The verdict column is filled by the oracle, not by the engine's self-report — the companion paper's `soft_delete` foil, whose own audit certifies a deletion its read path ignores, is the standing reason.

3.6 Scope of claims: what travels and what does not

Every run is stamped with a hardware fingerprint (platform, physical cores, RAM, library versions, both seeds, k). The experiments reported here were executed on a single workstation (macOS arm64, 10 physical cores), with both engines containerized on the loopback interface. Two claim classes follow, and we hold the distinction throughout:

Portable (hardware-independent): recall@k, leak rate, verdicts, `ops_until_gone`, prediction adjudications, mechanism evidence (plan text, model fits), and ratios computed within one fingerprint (storage amplification, relative build throughput, relative latency between engines in the same run). These reproduce exactly from the seeds.

Fingerprint-bound: absolute latency and throughput. All latencies are client-observed and therefore *transport-inclusive* (driver and loopback TCP); they are comparable across engines within one fingerprint, and the within-fingerprint comparisons are the only latency claims this paper makes. No absolute millisecond figure herein is a production estimate, and all are conditioned on dim=64: at production embedding dimensions (768–3,072) the exact-search baseline grows by roughly the dimension ratio and the case for approximate indexes strengthens accordingly; we state this conditioning wherever a conclusion could flip with it (§5.3, §8).

3.7 Registered predictions

Where a hypothesis preceded an experiment, its prediction was written into the experiment's driver before the run, and the adjudication — including three failures and one self-caught confound — is part of the published record (§5.2, §5.4, §6). We treat this as load-bearing methodology rather than ornament: §6 shows a case where it materially corrected the paper's own headline.

4 Guarantees: the inversion

The companion paper's question, carried to production engines: do commercial vector databases exhibit claims-but-leaks — advertising deletion or tenancy, accepting every call, returning success, and failing to enforce on the read path? Across two engines, four orders of corpus magnitude (4×10^3–10^6), supersession depths to 16, and every probe configuration in the study, the answer is no. Driven as documented, the engines are honest.

4.1 Deletion is enforced on read, immediately (W6)

In every 7a configuration at every scale, deleted records were unrankable at the first post-delete read: `ops_until_gone = 1`, leakage 0.000, with survivor recall intact against the post-delete oracle [F-PG-2, F-QD-4, F-S-5, F-S-9]. The result is non-trivial for both architectures, differently. pgvector deletes through MVCC row visibility: excision is on the read path by construction, with physical reclaim deferred to VACUUM — same verdict as an eager design, different bill, which is precisely the pairing the ledger exists to record. Qdrant deletes by marking points inside live HNSW segments, with physical reclaim deferred to the optimizer — the architecture class where a "deleted but still rankable" window would most plausibly live, and the reason it was instrumented first. With the graph confirmed live (`indexed_vectors_count` equal to the corpus on every row), deletes issued with `wait=true` were excluded by the read path before any reclaim occurred: gone at probe 1, leak 0.000, on indexes from 4×10^3 to 10^6 vectors. The claims-but-leaks failure mode did not reproduce on its likeliest commercial candidate.

Scope of the negative: the probe covers the steady-state read path. Windows it does not cover — optimizer mid-merge, and the durability boundary in which a snapshot/restore cycle revives a committed delete (the companion paper's `resurrecting` foil, one boundary over) — remain open and are named in §8.

4.2 Tenant scoping, driven correctly, never leaked (W5, posture 7a)

With the tenant predicate applied as documented, out-of-tenant leakage was 0.000 in every configuration, at every ef, at every N, on both engines [F-PG-1, F-QD-2, F-S-1, F-S-3]. The corpus makes this a real test: tenants are interleaved, so the unscoped nearest neighbors of every query are dominated by out-of-tenant vectors, and a store that ranked by distance alone would leak at roughly 7/8. What varies in 7a is not leakage but *recall* — on one engine, drastically (§5.1). The guarantee axis and the cost axis separate cleanly here, and the paper's title is this separation.

4.3 The failure mode is real — one layer up (posture 7b)

In the deployed posture — the tenant predicate omitted on read, nothing else changed — both engines return the global top-k: cross-tenant leakage 0.85–0.86, in-tenant recall 0.14, matching the companion paper's `leaky_tenant` foil almost exactly [F-PG-1, F-QD-2]. Three properties make this operationally serious. The failure is *silent*: no error, no warning. It is *full-looking*: result sets are well-formed and confidently ranked, with nothing in the response to distinguish them from

scoped results. And it leaves *no recall footprint on the leaking side* — the property the companion paper proved for its foils holds for production engines' deployment surface.

The precise statement matters, and we are careful with it: this is **not** claims-but-leaks. Neither engine claims to self-enforce tenancy; both enforce exactly the predicate they are given, and 7a passes. What the 7b posture establishes is that tenancy on both engines is *by convention* — a per-query opt-in, defeatable by a one-line omission in the calling layer. The companion paper located the failure mode in stores that ignore a scope they were handed; in production, the same observable failure arises one layer up, from a scope never handed to a store that would have honored it. Conformance testing that exercises only the engine-correct path cannot see this, which is the argument of §7 that guarantee verification belongs in deployment pipelines, not only engine test suites.

4.4 The other side fired once — and earned its keep

Across the entire study, the over-restriction verdict — the side of the two-sided doctrine that leak-only evaluations omit — fired on exactly one real configuration: Qdrant with graph traversal forced (full_scan_threshold lowered to 10 KB, a deliberately non-default setting) on the supersession corpus at depth 16, where filtered traversal returned recall 0.00 with leakage 0.00 at every ef [F-H4]. A leak-only suite would score those rows as its best possible result — zero leakage — while the store was returning nothing a user needed. We analyze the mechanism in §5.2; methodologically, the cell is the in-the-wild justification for scoring both directions, and for the rule that 0.000 leakage alone is never a pass.

4.5 The inversion, stated

On purpose-built foils, the companion paper showed that a capability claim does not certify behavior. On production engines, this study finds the contrapositive comfort and its limit: the engines certify their behavior — and the deployment surface does not. Leakage in this study was never an engine lying about a guarantee; it was an integration omitting one. What *does* differentiate the engines — sharply, by an order of magnitude or by the difference between an answer and no answer — is what each guarantee costs. The remainder of the paper measures that.

---

5 Costs: the divergence

5.1 Post-filter starvation: a mechanism, not an incident

pgvector applies scoping predicates *after* the ANN index scan: the HNSW traversal yields approximately ef candidates from the global corpus, and the predicate then discards the out-of-scope ones. Under a selective filter the expected scoped recall is therefore bounded by the survivors, recall ≈ min(1, ef·s/k) for selectivity s — a model stated before the mechanism probes and carried through three instruments [F-PG-6]:

1. **Plan text.** `EXPLAIN ANALYZE` on the scoped query at ef=16 shows an index scan with `rows=1, Rows Removed by Filter: 15`: sixteen candidates fetched,

fifteen discarded by the tenant predicate. Post-filtering, narrated by the engine itself.
2. **Forced-index probe** (n=4,000, s=1/8): scoped recall 0.25 / 0.54 at ef 16 / 40 against the model's 0.20 / 0.50. (At ef=64 the model's saturation point arrives early for this geometry — observed 1.00 vs predicted 0.80 — recorded as a model limitation.)
3. **Statistics-pinned ledger** (same corpus): scoped recall 0.23 / 0.50 / 1.00 / 1.00 across ef 16/40/64/128, matching the plan boundary cell for cell — the planner abandons the index for an exact filtered sort at ef ≥ 64 at this small scale.

That last detail is the scale trap. At n = 2×10^4 (s=1/64) the planner still rescued the highest-ef configuration by flipping to an exact scan over the ~312-row tenant subset (recall 1.00 at sub-millisecond latency). At 10^5 it stops flipping, and the engine as shipped tops out at scoped recall 0.04 / 0.10 / 0.31 across ef 16/64/256 (model: 0.025 / 0.10 / 0.40) [F-S-1]. At 10^6 the best swept configuration reaches 0.20 at 11.4 ms [F-S-6]. Post-filtering under selective filters is not *degraded* at scale; it is broken, and the small-N behavior that masks it is itself scale-dependent — a planner courtesy that quietly expires.

The engine's own mitigation, `hnsw.iterative_scan = relaxed_order`, restarts the traversal until enough survivors accumulate. It works where the problem is shallow — at n=4,000 it recovers ef16 scoped recall from 0.25 to 0.94 — and decays where it matters: across N = 2×10^4 / 10^5 / 10^6 its recall ceiling falls 0.99 → 0.81 → 0.43 while its price rises 5 → 8 → 30 ms, with the plateau flat across ef (the signature of a binding internal scan cap) [F-S-2, F-S-6]. The ledger sentence for the architecture: at 1/64 selectivity and 10^5–10^6 scale, correct scoping on a post-filter engine costs either approximately tenfold latency for partial recall, or the recall itself.

5.2 Planning is the architecture

Qdrant's filtered search is a per-segment *planning* decision: a cardinality estimate routes each segment either through filter-aware graph traversal or through an exact scan of the payload-matched subset, governed by `full_scan_threshold`. Three results characterize the planner, including one failed prediction that became the model.

The model earned out-of-sample status. Our initial, collection-level reading of the threshold predicted that default-configuration unfiltered recall would fall below 1.00 at N=10^5 (corpus ~26 MB > 10 MB threshold). The prediction failed — recall stayed 1.00, flat, at every ef. The refined, per-segment model (≈8 segments of ~3 MB each, all below threshold, hence exact per segment) fit every row at both N, and issued a registered forecast for 10^6: segments ~32 MB now exceed the threshold, so default unfiltered recall should finally drop below 1.00 and vary with ef, while the scoped query (cardinality ~4 MB per tenant) stays exact at 1.00. Observed at 10^6: unfiltered 0.14 / 0.37 / 0.69, ef-monotone; scoped 1.00, flat [F-S-0]. A failed prediction, refined, then confirmed out of sample — the arc we would want any benchmark's engine model to survive.

The contrast at matched conditions. On identical data, ef, and selectivity at 10^5 with traversal forced, the filter-aware graph holds scoped recall 0.91 at ef=16 where post-filtering starves to 0.04 — and shows the telling inversion that forced-

scoped (0.91) far exceeds forced-unfiltered (0.43): the filter *shrinks* a traversal-aware search problem rather than starving a traversal-blind one [F-S-3]. At 10^6 the default planner delivers the correct answer by the cheapest means available: scoped recall 1.00, flat, at 1.7–1.8 ms — an exact scan over the ~15.6k-row tenant subset, which is precisely what a competent plan looks like at that cardinality — while the post-filter engine at the same cell delivers 0.20 at 11.4 ms [F-S-6]. The mature statement of the axis: the engine that treats filtered search as a planning problem delivers exactness cheaply; the engine that bolts an ANN index onto a general-purpose planner delivers neither recall nor speed at this scale and selectivity.

The planner is load-bearing, not cosmetic. The same forced-traversal variant that demonstrates the graph's strength exposes its structural limit. On the supersession corpus at depth 16 — selectivity 1/16 with the valid records scattered one per tight near-duplicate cluster — forced filtered traversal collapses to recall 0.00 at every ef (§4.4), our hypothesized mechanism being effective disconnection of the filtered subgraph: traversal restricted to so sparse and so clustered a subset cannot reach the valid points [F-H4]. The engine's default configuration never enters this regime — its cardinality estimate routes such queries to the exact path — which reframes the fallback: it is not a performance courtesy but the component standing between the user and a structural failure mode of filtered graph traversal. We accordingly treat "do not lower full_scan_threshold in deployment" as a conformance-relevant configuration rule, not a tuning tip.

5.3 What scale actually costs (and for whom)

Three cost results frame engine selection, all conditioned on dim=64. Storage amplification on pgvector — total relation size over raw vector bytes, covering heap overhead, the HNSW graph, and the scope index — is 3.70× / 3.67× / 3.66× at $N = 4\times10^3$ / 10^5 / 10^6: stable across three orders of magnitude, and measured exactly [F-PG-5, F-S-4, F-S-8]. Build throughput diverges where storage does not: pgvector falls off a cliff from 8.2k to 1.33k vectors/s between 10^5 and 10^6 (12.5 minutes for the million-row index, and only after raising container shared memory, since a default 64 MB /dev/shm cannot host a ~512 MB parallel HNSW build), while Qdrant declines gently from 19.7k to 15.6k [F-S-8]. And the exact-search baseline refuses to lose at this dimension: in-process brute force scores 1.00 at 10.7 ms at 10^6, while pgvector's best swept unfiltered configuration scores 0.47 at 14.3 ms — at dim=64, ANN's reason to exist has not yet arrived at a million vectors, and any "architectural corner" claim below that scale would be premature [F-S-7]. We state the conditioning rather than the temptation: at production dimensions the exact baseline inflates by roughly the dimension ratio, and the approximate index's case strengthens; the corner is real but cost-shaped, and one tested engine already stands outside it.

5.4 The supersession tax is twofold — and was mispredicted

Agent memory under supersession stores every version of a fact as a row, with validity intervals, and answers `as_of` queries by temporal predicate. Our registered model priced this as pure post-filter starvation: recall ≈ min(1, ef/(D·k)) at depth D. The model was confirmed in direction and wrong in size, because version-as-row pays *twice*. At D=4 the as_of recall curve (0.20 / 0.44 / 0.82 across ef 16/64/256) matches the engine's *unfiltered* recall at the same physical N — the

inflation of the corpus by stale versions degrades graph recall before the validity filter starves anything. At D=16 both channels compound: 0.04 / 0.12 / 0.31, beneath even the naive model, with the iterative-scan mitigation plateauing at 0.29–0.40 [F-H4]. Temporal leakage, meanwhile, was 0.000 in every cell of the temporal study — the predicates enforce validity perfectly; what they cannot do on this architecture is afford it. (The companion result on Qdrant is the subject of §6.)

5.5 Every gap closed at the adapter layer

The decisive question for the architecture of governed memory is whether these costs require engine-native remediation — first-class temporal semantics inside the store — or close above the engine. Both engines closed, each by a different one-screen fix, and we registered gate criteria before running either probe.

Epoch partitioning (pgvector). The adapter maintains one HNSW table per validity epoch plus a current-snapshot table selected by exact match on the open-interval sentinel; `as_of t` routes to epoch table ⌊t/E⌋, with the full validity predicate retained in every query so that routing is an optimization and the predicate remains the correctness guarantee. Registered gate: recall ≈ the depth-1 baseline independent of D; latency flat in D; storage tax ≈ +1/D with total build not exceeding monolithic. Observed at D=16: as_of recall 0.42 / 0.75 / 0.97 against the D=1 baseline of 0.41 / 0.74 / 0.98 — statistically indistinguishable at sixteen times the history — at p50 0.6–1.8 ms, versus the monolithic 0.04–0.31, with leakage 0.000 throughout; storage tax +25% at D=4 and +6% at D=16, tracking +1/D as predicted. The build half of the gate *failed*: 59.6 s versus 25.2 s monolithic (~2.4×, per-table index overhead), a bounded cost but a wrong prediction, recorded as such [G3].

Payload range indexing (Qdrant). §6 describes how this probe also corrected our own record; the result here: integer payload indexes on the two validity fields take the monolithic engine to as_of recall 1.00 at 1.5–3.4 ms, flat in D — no partitioning machinery at all. A collection-per-epoch variant matches it (1.00 at 1.5–2.6 ms) while paying 3.5× the build and +6% storage, and is therefore unnecessary on this engine [F-Q2-1].

The cheapest sufficient fix differs per engine — partitioning on the post-filter architecture, an index declaration on the planning architecture — and both live in the adapter. The conclusion the gate was built to decide: native bi-temporality, the one capability gap that ever pointed toward purpose-built storage in this study, is an optimization, not a requirement.

---

6 The confound we caught

One number in this study was wrong by more than an order of magnitude, the error was ours, and a registered probe found it. We report the episode in full because it is the strongest evidence the paper contains for its own methodology.

The temporal study (§5.4) initially recorded Qdrant's default configuration delivering exact as_of answers (recall 1.00, leakage 0.000) at a price that grew linearly with supersession depth: p50 10.5 → 24.9 → 65.1 ms across D = 1 / 4 / 16,

with the open-interval probe costlier still (83–84 ms at D=16). We drew the natural conclusion — exactness via fallback scan scales with the version bloat — and that conclusion briefly stood as the study's strongest argument that emulated bitemporality is expensive everywhere, the finding most favorable to engine-native temporal storage.

The conclusion did not survive its own follow-up. Designing the partitioning probe for Qdrant forced an audit of the adapter's index provisioning, which revealed that our adapter had never created payload indexes on the two validity fields: every temporal predicate was being evaluated by unindexed scan. We registered the cheap fix as a competing hypothesis *before* running either variant — if integer range indexes alone recovered low-millisecond exactness, the 65 ms figure was an instrument artifact, not an engine cost. They did: recall 1.00 at 1.5–3.4 ms at D=16, roughly 95% of the reported cost gone with two index declarations [F-Q2-0]. All affected rows in the published ledgers carry a confound flag; the erratum strengthens rather than weakens the paper's conclusion (the gap closes even more cheaply than the gate assumed), but it would have inverted the §5.5 comparison had it shipped.

Three properties of the methodology made the catch, and we believe each is necessary. The *registered prediction* forced the cheap explanation to be tested before the expensive conclusion was published. The *verdict-paired ledger* preserved the provenance of every figure, so the correction was a flagged row rather than a silent re-run. And the *adapter-as-instrument* framing kept our own integration inside the test boundary — which is, in miniature, the paper's deployment-surface thesis from §4.3: the engine was honest; the layer driving it was the defect. A benchmark whose own harness is outside its audit scope will, sooner or later, publish its harness as an engine result. Ours nearly did, and the record of *nearly* is, we contend, worth more to the reader than a clean table.

---

7 Implications

7.1 Conformance belongs in deployment pipelines

The study's central operational lesson follows directly from §4: a 7a pass certifies the engine; only a 7b probe certifies the deployment. Every leakage event we observed was produced by an integration omitting a predicate that an honest engine would have enforced — silently, with full-looking results and no recall footprint. An engine-level conformance suite, however rigorous, runs the engine-correct path by construction and is therefore blind to the entire class. The practice this implies is concrete: run the two-sided suite against the *deployed configuration* — the actual adapter, the actual collection settings, the actual provisioning code — in both postures, as an acceptance gate in the deployment pipeline, and re-run it on configuration change. The harness published with this paper is that gate; its adapters are the seeds of production ones.

The structural complement to the gate is to remove the omission class by construction: an adapter protocol that requires the scope at the type level — in which an unscoped read is unrepresentable rather than merely discouraged — converts tenancy from a per-query convention into a property of the integration. (We operate one such layer in production, atop the engines studied here; this paper's results

are the evidence base for that design rather than an evaluation of it.)

7.2 For protocol design: configuration is conformance-relevant

The companion paper argued that a memory protocol must ship an executable conformance suite because a declared capability is not a guarantee. The present study sharpens that requirement in two ways. First, *"supports X" must be tested per-deployment, not per-engine*: the same engine that passes W5 in posture 7a fails it in 7b, so a conformance stamp attached to an engine name, rather than to a deployment configuration, certifies the wrong object. Second, *specific configuration rules are themselves conformance-relevant*, because we observed configurations that convert a passing system into a failing one without touching application code: index provisioning that omits a payload index on a filterable field multiplies the cost of a correct temporal answer by ~30–40× (§6); lowering a planner threshold that looks like a performance knob (`full_scan_threshold`) exposes a structural traversal-collapse regime that the default planner exists to prevent (§5.2). A protocol's conformance suite should therefore pin and test the configuration surface — indexes present on every filterable field, planner guards at defaults — exactly as it tests the operation semantics.

7.3 For engine selection: buy the architecture, then pay its specific bill

The ledger's selection lesson is that the *filtering architecture class* predicts the cost of correctness better than any single benchmark number. At matched data, parameters, and selectivity, the post-filter engine's scoped recall at 10^6 was 0.20 at 11.4 ms against the planning engine's 1.00 at 1.7–1.8 ms — not a tuning gap but an architectural one, with a confirmed mechanism on each side. Comparisons should be made at matched recall on swept frontiers, never between single points; and the remediation budget differs per architecture in kind, not just size:

architecture (engine studied)	binding cost	cheapest sufficient fix (measured)
post-filter (pgvector)	scoped/temporal recall starvation; build cliff at 10^6	epoch-partitioned adapter: 0.97 @ 1.8 ms at D=16 (+1/D storage, 2.4× build)
planning (Qdrant)	none observed at defaults; temporal cost was an indexing fault (ours)	payload range indexes provisioned in code: 1.00 @ ~2 ms, flat in D

Both fixes live above the engine. Which is the study's last implication:

7.4 For governed memory storage: the gap did not require an engine

This research program began from the hypothesis that governed agent memory might require a purpose-built storage engine — native bi-temporality, native scoping, native erasure attestation. The evidence ran the other way at every gate. The guarantees that exist hold; the guarantee that is genuinely absent from the engine layer (cryptographic operation receipts and erasure certificates) is, on our survey

to date, absent from *every* engine and already implementable above the adapter; and the one capability gap that carried observed cost — temporal semantics under supersession — closed at the adapter layer on both architectures against pre-registered gates (§5.5). We state the reopening conditions rather than foreclose them: a demonstration that attestation is materially superior below the adapter; a durability-boundary failure (a committed delete reviving across snapshot/restore) that adapter-level guarantees cannot survive; or the partitioning pattern failing on non-aligned production interval distributions. Absent those, the economical conclusion stands: the layer, the gate, and the ledger — not a new engine — are what governed agent memory was missing.

---

8 Limitations (skeleton, to finalize after §7/§9)

Synthetic corpora at dim=64. All corpora are procedurally generated at embedding dimension 64. Recall, leakage, verdicts, mechanism chains, and predicted-vs-observed adjudications are dimension-robust in direction; absolute latency and the exact-vs-ANN crossover point are not. At production dimensions (768–3,072) the exact baseline inflates roughly with the dimension ratio and ANN's case strengthens; every §5.3 corner statement is conditioned accordingly. A rerun at production dimensions with server-side timing is the designated journal extension (reproduction appendix).

Transport-inclusive, single-fingerprint latency. All latencies are client-observed over loopback against containerized engines on one workstation. Within-fingerprint engine comparisons are valid; absolute milliseconds are not production estimates and are never presented as such.

Two engines. pgvector and Qdrant were selected as locally-runnable representatives of the post-filter and planning architecture classes. The findings are architecture-class claims supported by one engine each; managed/serverless engines (different control planes, namespacing, and deletion machinery) are characterized in §9 by documentation only.

Favorable interval geometry for partitioning. The supersession corpus has epoch-aligned validity intervals — the best case for epoch partitioning. Arbitrary intervals force multi-epoch placement (bounded near 2× for chain-shaped histories); the payload-index fix (§5.5) carries no such dependence. Qdrant's temporal exactness rides on per-segment valid-set cardinality staying below the full-scan threshold and requires re-verification at production scale.

Untested windows. Optimizer mid-merge deletion behavior; the snapshot/restore durability boundary (a committed delete reviving on restore); concurrency under load. Each is named rather than waved at, and each has a designed probe in the harness backlog.

Forced configurations are labeled. The OVER_RESTRICTS result (§4.4) and the traversal-collapse mechanism (§5.2) arise under a deliberately non-default `full_scan_threshold`; no shipped default exhibited them. They are findings about what the default configuration protects against, not about engine misbehavior.

---

9 Related Work

Agent-memory systems and their evaluation. MemGPT/Letta (Packer et al., 2023), Zep's bi-temporal knowledge graph (2025), and mem0 (2025) are representative of the layer that sits *on* the engines studied here and inherits their guarantees while adding scoping, summarization, and forgetting logic of its own; the companion paper situates them in detail. Dominant memory evaluations (LOCOMO, Maharana et al., 2024; LongMemEval, Wu et al., 2024) score end-to-end QA with model judges and test no privacy boundary; the companion paper's oracle-grounded, two-sided design is the methodological alternative this study carries to production engines. To our knowledge — and we make the claim no more strongly than that — no prior work has driven commercial vector databases through a two-sided, oracle-grounded deletion-and-tenancy conformance suite; published engine comparisons are overwhelmingly recall/latency benchmarks (the ann-benchmarks tradition) or feature surveys.

Filtered approximate search. The starvation mechanism of §5.1 and the traversal-collapse regime of §5.2 are instances of the filtered-ANN problem studied by Filtered-DiskANN (Gollapudi et al., 2023), ACORN (Patel et al., 2024), and successors (e.g., SIEVE, Li et al., 2025; see Compass, 2025, for a current survey). Our contribution to that literature is not an algorithm but field evidence: which architecture classes expose the problem to the user, at what measured cost, and what a deployment can do about it.

Industry architecture survey (current to June 2026, vendor primary sources; full source-tagged table in the artifact repository). Two axes matter for governed memory, and the swept engines arrange cleanly along both. *Filtered-search routing:* every server-class engine swept has converged on some form of cardinality-routed filtering — Qdrant's per-segment full-scan threshold (measured here), Weaviate's inverted-index allow-list with a brute-force route below a cutoff and an ACORN-based strategy above it, Vespa's pre-filter-by-default with automatic exact fallback below an approximate-threshold, Milvus's partition-key index routing, and Azure AI Search's user-visible per-query filter-mode knob — while pgvector (an ANN index under a general-purpose SQL planner) and the bare libraries (FAISS, hnswlib, usearch) leave the problem to the caller. Our two instrumented engines are therefore fair representatives of the two poles of a documented industry axis, and the planner-class behavior we measured generalizes in kind, though not in numbers, across the class. *Tenancy:* the market spans a spectrum on which the §4.3 omission class weakens by degrees — filter-convention (pgvector, Qdrant, Milvus filter-mode), namespace/partition (Pinecone namespaces; Milvus partition key; Turbopuffer's namespace-per-prefix on object storage), shard-per-tenant with a mandatory per-operation tenant key (Weaviate), and scope-first, index-free exact search over per-user co-located data (Vespa streaming mode — whose design argument, that approximate indexes are ill-suited to strongly-filtered personal-data search, is independently consonant with our F-S finding that exact search wins at scoped cardinalities). A per-call scope argument exists in all but the last, so the deployment-surface lesson of §7.1 applies, in weakening degrees, across nearly the entire market.

Attestation. On a verified sweep of vendor documentation current to June 2026: operational audit logs exist across the market (largely enterprise-tier, batched, mutable, retention-bound — e.g., Pinecone's ~30-minute-batched control-plane logs to customer S3; Weaviate's RBAC decision logs; Milvus/Zilliz access and

audit logs; Qdrant Cloud's recently added audit logs), while cryptographic per-operation artifacts — signed operation receipts, erasure certificates — appear in no engine's documentation. The nearest neighbors each miss by one axis: disk-wipe certificates attest media, not records; TEE-based confidential vector search attests the platform, not the operation; third-party embedding signing attests stored vectors, not operations. The gap is categorical rather than cost-shaped, and it is implementable above the adapter — which is where the temporal gap also closed (§5.5).

Temporal data management. The bi-temporal model our supersession corpus encodes is classical (Snodgrass, 1999; SQL:2011); what this study adds is its measured price on vector-search architectures that lack it, and the demonstration that epoch partitioning and validity-field indexing — both standard relational techniques — recover the ideal at the adapter layer.

Benchmark methodology. The verdict-paired ledger, registered predictions, and the published self-confound (§6) sit in the tradition of reproducible performance evaluation (the TPC lineage; ann-benchmarks' pinned-environment discipline) and extend it with a practice we argue is load-bearing for conformance work specifically: keeping the harness inside the audit scope.

---

10 Artifacts and Reproduction

The artifact repository is archived on Zenodo — version of record for this paper: DOI 10.5281/zenodo.20666339 (v1.0.0); concept DOI resolving to the latest version: 10.5281/zenodo.20666338 — and developed at github.com/cayerbe/grafomem-db-bench. It contains the complete instrument and record: the harness (adapter protocol, exact-KNN and bi-temporal oracles, two-sided metrics, checkpointed drivers for the conformance, scale, supersession, and partitioning studies), the two engine adapters plus the partitioned variants, all five canonical ledgers (conformance, scale, supersession, partitioning gate, and the Qdrant probe) in CSV and JSON with embedded hardware fingerprints, the source-tagged landscape table, the verified attestation sweep, and the hypothesis registry. Corpora are regenerated from pinned seeds (1729/4104); recall, leakage, verdicts, and ops_until_gone reproduce exactly; latency re-measures within the reader's own fingerprint. Engines run from the included compose file (pgvector on a loopback port with shm_size raised for parallel HNSW builds; Qdrant with tmpfs-ephemeral storage so every run starts clean). Each experiment is one driver invocation; drivers resume from checkpoints. The repository's commit history is part of the artifact: every registered prediction is committed before the run that adjudicates it, including the three that failed and the confound of §6.

Tables

Source ledgers: `research/rq9-cost-ledger/results/{cost,scale,h4,g3,g3q}_ledger.csv` and `research/rq7-conformance/commercial_ledger.csv`. Every cell verifiable via `harness/rq9/audit_numbers.py`. Shared fingerprint family: macOS arm64, 10

physical cores, dim=64, k=10, seeds 1729/4104; all latencies client-observed, transport-inclusive; engines containerized on loopback.

Table 1 — Two-sided conformance, both postures (§4)

n=4,000 · 8 interleaved tenants · per-engine best swept configuration.

engine	posture	W5 scoped recall	W5 leak	W5 verdict	W6 leak	W6 gone@	W6 verdict
pgvector	7a engine-correct	1.00	0.00	PASS	0.00	1	PASS
pgvector	7b as-deployed	0.14	0.85–0.86	**LEAKS**	0.00	1	PASS[1]
Qdrant	7a engine-correct	1.00	0.00	PASS	0.00	1	PASS
Qdrant	7b as-deployed	0.14	0.85–0.86	**LEAKS**	0.00	1	PASS[1]

Caption: The inversion and the relocation in one table: driven as documented (7a), both engines hold both boundaries; with the tenant predicate omitted on read (7b) — one integration line — both silently return the global top-k, with full-looking results and no error. [1]Deletion is adjudicated by its own leak column; 7b's depressed deletion-row recall is cross-scenario contamination from the isolation failure, adjudicated by the W5 row. (Fingerprint-bound: none — all cells in this table are portable.)

Table 2 — Post-filter starvation: the mechanism (§5.1)

pgvector, n=4,000, tenant selectivity 1/8, statistics pinned (ANALYZE).

instrument	ef16	ef40	ef64	ef128
candidate-budget model min(1, ef·s/k)	0.20	0.50	0.80[2]	1.00[2]
forced-index probe (observed)	0.25	0.54	1.00	—
engine-as-shipped ledger (observed)	0.23	0.50	1.00[3]	1.00[3]
unfiltered baseline (observed)	0.67	0.89	0.96	0.99
+ iterative_scan mitigation	0.94	—	—	—

Caption: Three independent instruments agree. Plan text at ef16: `Index Scan … rows=1, Rows Removed by Filter: 15` — sixteen candidates fetched, fifteen discarded post-scan. [2]Model saturates early for this geometry (recorded limitation). [3]Planner abandons the index for an exact filtered sort at ef≥64 at this small scale — the escape-hatch that closes at 10^5 (Table 3). Recall cells portable; dim=64 conditioning applies to any latency reading.

Table 3 — Scale: the divergence (§5.1–5.3)

Selectivity 1/64 (64 tenants) · ef ∈ {16, 64, 256} · scoped recall@10 [@ p50 ms where stated].

engine / variant	$N=2\times10^4$	$N=10^5$	$N=10^6$
pgvector (shipped)	1.00 (planner→exact)	0.04 / 0.10 / 0.31	best 0.20 @ 11.4 ms (ef256)
pgvector + iterative_scan (ceiling @ price)	0.99 @ 5 ms	0.81 @ 8 ms	0.43 @ 30 ms
Qdrant (default planner)	1.00 flat	1.00 flat	**1.00 flat @ 1.7–1.8 ms**
Qdrant (forced traversal, non-default)	—	0.91 (ef16; vs unfiltered 0.43)	—

Supporting cost rows, same fingerprint:

measure	pgvector	Qdrant	oracle (exact, in-process)
build rate $10^5 \rightarrow 10^6$ (vec/s)	8.2k → **1.33k**	19.7k → 15.6k	—
storage amplification (4×10^3 / 10^5 / 10^6)	3.70× / 3.67× / 3.66×	estimate-class	1× (raw)
unfiltered @ 10^6, ef256	0.47 @ 14.3 ms	0.69	1.00 @ 10.7 ms

Caption: Post-filtering is not degraded at scale — it is broken, and the small-N planner rescue is itself scale-dependent. The mitigation's recall ceiling decays as its price grows. The planning engine delivers exactness at the cheapest correct plan (~15.6k-row exact tenant scan). Forced traversal is a deliberately non-default configuration. Latencies fingerprint-bound, transport-inclusive, dim=64 (at production dims the exact baseline inflates ~with the dimension ratio and ANN's case strengthens).

Table 4 — The supersession tax (§5.4)

20k chains × D versions (drift 0.15) · as_of mid-history · recall@10 [@ p50 ms].

variant	D=1 (20k rows)	D=4 (80k)	D=16 (320k)
pgvector	0.42 / 0.75 / 0.99	0.20 / 0.44 / 0.82	0.04 / 0.12 / 0.31 @ 0.8–2.4 ms
pgvector + iterative_scan	0.39 / 0.75 / 0.98	0.44 / 0.53 / 0.86	0.29 / 0.31 / 0.39

variant	D=1 (20k rows)	D=4 (80k)	D=16 (320k)
Qdrant (default) †	1.00 @ 10.5 ms	1.00 @ 24.9 ms	1.00 @ 65.1 ms (now-probe 83–84 ms)
Qdrant (forced traversal, non-default)	0.58 / 0.93 / 1.00	0.06 / 0.21 / 0.52	**0.00 / 0.00 / 0.00 — OVER_RESTRICTS**

Caption: Cells are ef 16/64/256. The twofold tax: D=4's curve matches the engine's *unfiltered* recall at comparable physical N (N-inflation degrades the graph before the validity filter starves). Temporal leakage 0.000 in every cell. † CONFOUND FLAG (F-Q2-0): the default-Qdrant latencies in this table were produced with the validity fields unindexed by our adapter — ~95% of the cost is instrument artifact, corrected in Table 5; recall cells unaffected. The forced-traversal collapse is the study's only OVER_RESTRICTS firing and arises under a deliberately non-default threshold.

Table 5 — Adapter-layer closures: the decisive cells (§5.5–§6)

D=16 (320k physical rows) · as_of · ef256 unless noted · one fingerprint family.

approach	recall	p50	build (D=16)	storage note
pgvector monolithic	0.31	2.4 ms	25.2 s	3.66–3.70× raw
pgvector + iterative_scan	0.39	2.2 ms	—	—
Qdrant unindexed-validity (H4, confound-flagged)	1.00	64–84 ms	14.6–15.7 s	estimate-class
pg-partitioned (epoch + sentinel snapshot)	**0.97**	**1.8 ms**	59.6 s (2.4×, GP3-half failed)	+1/D placement (+6% @ D=16)
Qdrant + validity payload indexes	**1.00**	**1.5–3.4 ms flat in D**	18.2 s	estimate-class
Qdrant collection-per-epoch (moot)	1.00	1.5–2.6 ms	63.7 s (3.5×)	+6%

Gate adjudication (registered before the runs): GP1 PASS — partitioned D=16 recall 0.42/0.75/0.97 ≈ D=1 baseline 0.41/0.74/0.98, depth-independent; GP2 PASS — p50 0.6–1.8 ms flat in D; GP3 split — storage +1/D as predicted (D=4 +25%, D=16 +6%), build half FAILED (2.4×, owned). QP0 — payload indexes alone recover exactness: the H4 row above carries the confound.

Caption: The cheapest sufficient fix differs per architecture — partitioning on the post-filter engine, two index declarations on the planning engine — and both live in the adapter. Native bi-temporality: optimization, not requirement. Latencies fingerprint-bound, dim=64; partitioning measured on epoch-aligned intervals (favorable case, ~2× placement bound general).

References

Format: author-year keys as used in DRAFT.md; convert to BibTeX at LaTeX time.

[companion] Ayerbe Posada, C. (2026). *GRAFOMEM: A Two-Sided, Oracle-Grounded Benchmark for Governed Agent Memory* (grafomem-bench v0.2.0). academia.edu/167598244. — link the Zenodo DOI of this paper reciprocally.

Agent memory systems - Packer, C. et al. (2023). *MemGPT: Towards LLMs as Operating Systems.* arXiv:2310.08560. - Rasmussen, P. et al. (2025). *Zep: A Temporal Knowledge Graph Architecture for Agent Memory.* arXiv:2501.13956. - Chhikara, P. et al. (2025). *mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory.* arXiv:2504.19413.

Memory evaluation - Maharana, A. et al. (2024). *Evaluating Very Long-Term Conversational Memory of LLM Agents* (LOCOMO). ACL 2024. □ confirm arXiv id 2402.17753. - Wu, D. et al. (2024). *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.* arXiv:2410.10813.

Filtered approximate search - Gollapudi, S. et al. (2023). *Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters.* WWW 2023. - Patel, L. et al. (2024). *ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data.* SIGMOD 2024. □ confirm arXiv id 2403.04871. - Li, Z. et al. (2025). *SIEVE.* □ resolve full citation (referenced via Compass survey). - (survey) *Compass: General Filtered Search across Vector and Structured Data.* arXiv:2510.27141 (2025/26).

Benchmarks & methodology - Aumüller, M., Bernhardsson, E., Faithfull, A. (2020). *ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms.* Information Systems 87. - TPC. *TPC Benchmark specifications* (lineage citation in §9; cite the TPCTC venue if submitted there).

Temporal data & deletion - Snodgrass, R. T. (1999). *Developing Time-Oriented Database Applications in SQL.* Morgan Kaufmann. - ISO/IEC 9075:2011 (SQL:2011) — system-versioned and application-time tables. - Bourtoule, L. et al. (2021). *Machine Unlearning.* IEEE S&P 2021. arXiv:1912.03817.

Industry primary sources (cited in §9 / RQ1 table; URLs + access dates live in RQ1_TABLE.md and H8_SWEEP.md) - Pinecone, Weaviate, Milvus/Zilliz, Qdrant, Vespa, Turbopuffer, Elastic/OpenSearch, Azure AI Search vendor documentation, accessed 2026-06-12.